Share

NextSeason(5).zip
Zip Archive

Please review the project and call out any problems and any missing items for MVP

Stopped thinking

I reviewed the current project snapshot as an MVP, not as a production v1. Overall, this is in surprisingly good shape. It feels like a complete Slice 2 implementation rather than a partially-finished prototype.

What Looks Complete

The MVP requirements in your docs appear to be implemented:

Search

- Search by title
- TVMaze integration
- Result list
- Loading, empty, and error states
- Show artwork and status

Show Details

- Detail screen
- Metadata display
- Next-season calculation
- Human-readable status presentation
- Track/untrack actions

Watchlist

- Add/remove shows
- Duplicate prevention
- SwiftData persistence
- Watchlist screen
- Pull-to-refresh

Notifications

- Permission flow
- Notification delivery
- Duplicate prevention logic
- Notification routing back into the app

Background Updates

- BGTaskScheduler integration
- Refresh service

- Change detection
- Notification generation

Engineering Quality

- Repository abstraction
- DTO/domain/persistence separation
- Extensive unit tests
- Environment injection
- Good separation of concerns

This is substantially cleaner than most portfolio projects I review.

Issues I Would Fix Before Calling MVP "Done"

1. TVMaze Attribution Visibility

Your decision log correctly notes that TVMaze attribution is required.

I can see attribution code referenced in `ShowDetailView`, but I would verify:

- Attribution is visible to users
- Attribution exists wherever TVMaze data is presented
- Attribution survives App Store review scrutiny

This is probably a small polish item, but it's important.

---

2. Background Refresh Expectations

This is more of a product issue than a code issue.

The marketing message must never imply:

> "We'll notify you immediately."

Because BGTask refreshes are best-effort.

I would explicitly say:

> "NextSeason periodically checks for updates and will notify you when changes are discovered."

You already understand this technically, but product wording matters.

---

3. Notification Permission Timing

Current flow appears to be:

1. User tracks show
2. Prompt appears

That's reasonable.

However I'd verify the experience carefully:

- User tracks first show
- Gets one permission prompt
- Never gets nagged repeatedly

If that's already true, I wouldn't change anything.

---

4. No Explicit Last Refresh Indicator

I saw references to `lastCheckedAt` and refresh timestamps in the model.

I did not immediately see a user-facing:

> Last updated: June 17, 2026

display.

This is not required for MVP, but it would increase trust significantly.

If it only takes an hour or two, I'd add it.

Things I Would NOT Add

These are classic scope-creep traps:

❌ Login

❌ Sign in with Apple

❌ Cloud sync

❌ Streaming availability

❌ Episode tracking

❌ Ratings/reviews

❌ Favorites beyond watchlist

❌ User profiles

You made the correct decision to defer all of them.

One Missing Portfolio Item

This is the biggest thing I noticed.

No Privacy Policy

If you intend to ship this to the App Store, even as a portfolio app:

- Create a simple Privacy Policy page
- State that data is stored locally
- State that TV show information comes from TVMaze
- State that no personal data is collected

This can literally be a one-page Markdown document converted to HTML.

The app itself doesn't need an in-app screen for MVP, but you'll want the document before App Store submission.

MVP Readiness Score

Product Scope

9.5 / 10

Very disciplined. You resisted all the common traps.

Architecture

9 / 10

Appropriately engineered without becoming enterprise nonsense.

Portfolio Value

9 / 10

The repository demonstrates:

- SwiftUI
- Networking
- Persistence
- Background processing
- Notifications
- Testing
- AI-assisted development workflow

which is exactly what you set out to showcase.

MVP Completion

If I were acting as product manager, I would say:

Feature-complete MVP.

Before shipping publicly I would add:

1. Verify TVMaze attribution
2. Add Privacy Policy
3. Consider a "Last Updated" timestamp

After those, I'd stop building features and move into Phase 3 polish/testing/App Store preparation.

The "Updated" timestamp is part of WatchlistRow. Is that sufficient?

Yes, assuming the timestamp in `WatchlistRow` is clearly visible and understandable to a normal user, I would count that as sufficient for MVP.

What I was looking for was a way for a user to answer:

> "How fresh is this information?"

If each tracked show shows something like:

- Updated today
- Updated yesterday
- Updated Jun 12

then you've already solved that problem.

The only thing I'd double-check is **what the timestamp actually means**.

There are two different timestamps that could exist:

1. **When TVMaze data for this show was last refreshed** ← useful
2. **When the user added the show to their watchlist** ← not useful for freshness

If your `Updated` field is #1, then I think you're fine.

One minor UX question I'd ask myself:

When looking at a watchlist with 20 shows, will users understand that "Updated" means "last checked against TVMaze"?

If not, you might eventually rename it to something slightly clearer:

- Last Checked
- Last Updated
- Checked Jun 17

But that's polish, not MVP.

So I'd remove that item from my "missing MVP" list unless the timestamp is actually tracking something other than refresh activity.

Do we currently have any written documentation for post-MVP tasks?

Stopped thinking >

Yes, but only at a very high level.

Yes, but only at a very high level.

The main document is:

`Documentation/MVPBacklog.md`

It currently contains three sections:

P0 – Required for MVP

The features you've been building toward.

P1 – Nice to Have

- Sort watchlist
- Filter watchlist
- Search within watchlist
- Better loading states
- Empty-state illustrations
- Enhanced animations

P2 – Future Versions

- User accounts
- Login/password reset
- Cloud sync
- Multi-device support
- Streaming provider information
- Social features
- Monetization

The problem is that this backlog was written very early in the project and hasn't really evolved along with our discussions. A few examples:

- We explicitly decided **not** to have accounts in the MVP, but the backlog doesn't explain *why* or what would trigger revisiting that decision.
- There is no mention of several ideas we've discussed, such as:
  - App Store release preparation
  - Privacy policy
  - Analytics (if ever desired)
  - Better notification controls
  - "Last checked" visibility improvements
  - Additional season-status intelligence
  - Widget support
  - WatchOS support
  - TV tracking integrations
  - Improved background refresh strategies

So the answer is:

Yes, there is a post-MVP backlog (`MVPBacklog.md`), but it is very lightweight and not something I would consider a real roadmap.

If I were maintaining this project, the next thing I'd probably add after declaring MVP complete is a new

document:

`Documentation/PostMVPRoadmap.md`

with sections such as:

1. App Store Readiness
2. MVP Polish
3. User-Requested Features
4. Growth Features
5. Explicitly Deferred Ideas
6. Nice Portfolio Enhancements

That would give you a much clearer place to capture ideas without accidentally expanding MVP scope